

R workshop part 4 - Data tidying

Jan Gačnik

Prepared: 2026-07-30

Data tidying in R

Happy families are all alike; every unhappy family is unhappy in its own way. — Leo Tolstoy

Tidy datasets are all alike, but every messy dataset is messy in its own way. — Hadley Wickham

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

variables

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

observations

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	216766	128042583

values

Source: Hadley Wickham, *R for Data Science*

Data tidying in R

What makes a dataset **tidy**?

- Each **variable** represents a **column**
- Each **observation** represents a **row**
- Each **value** is a **cell**; each cell containing a single value
- *Variable names* are descriptive: e.g. `temperature` and not `t1`
- *Missing values* are treated consistently: e.g. `NA` only, without `NULL`, `-9999`, `none`, etc.

Tidy data

Let's take a look at some example datasets that demonstrate both messy and tidy data, again coming from the `tidyverse` package:

```
1 library(tidyverse)
2 table1
```

```
# A tibble: 6 × 4
  country    year cases population
  <chr>    <dbl> <dbl>      <dbl>
1 Afghanistan 1999    745  19987071
2 Afghanistan 2000   2666  20595360
3 Brazil      1999   37737  172006362
4 Brazil      2000   80488  174504898
5 China       1999  212258  1272915272
6 China       2000  213766  1280428583
```

```
1 table2
```

```
# A tibble: 12 × 4
```

```
  country    year type      count
  <chr>    <dbl> <chr>      <dbl>
1 Afghanistan 1999 cases        745
2 Afghanistan 1999 population 19987071
3 Afghanistan 2000 cases        2666
4 Afghanistan 2000 population 20595360
5 Brazil      1999 cases        37737
6 Brazil      1999 population 172006362
7 Brazil      2000 cases        80488
8 Brazil      2000 population 174504898
9 China       1999 cases        212258
10 China       1999 population 1272915272
11 China       2000 cases        213766
```

Tidy data

Let's take a look at some example datasets that demonstrate both messy and tidy data, again coming from the `tidyverse` package:

```
1 library(tidyverse)
2 table1
```

```
# A tibble: 6 × 4
  country      year cases population
  <chr>      <dbl> <dbl>      <dbl>
1 Afghanistan 1999     745   19987071
2 Afghanistan 2000    2666   20595360
3 Brazil       1999   37737   172006362
4 Brazil       2000   80488   174504898
5 China        1999  212258  1272915272
6 China        2000  213766  1280428583
```

```
1 table3
```

```
# A tibble: 6 × 3
```

```
  country      year rate
  <chr>      <dbl> <chr>
1 Afghanistan 1999 745/19987071
2 Afghanistan 2000 2666/20595360
3 Brazil       1999 37737/172006362
4 Brazil       2000 80488/174504898
5 China        1999 212258/1272915272
6 China        2000 213766/1280428583
```

Benefits of tidy data

- Easier data exploration
- Simplified data transformation and visualization
- Improved reproducibility
- Many R functions work easily with tidy data formats

Data pivoting

Unfortunately, real-life datasets are messy, because:

- Data is usually organized with some goal in mind other than analysis, e.g. data entry, human readability, storage, ...
- Most people are not familiar with principles of tidy data

We will learn how to **pivot** the data into a tidy form by *lengthening* or *widening*.

Data lengthening

First, we will learn data lengthening using `pivot_longer()` function

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

Pivot data longer

```
data %>%
  pivot_longer(
    cols = 1999:2002,
    names_to = "year",
    values_to = "cases"
  )
```



country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

Source: Batra, Neale, et al. *The Epidemiologist R Handbook*. 2021

Data lengthening

We will use the `billboard` dataset to demonstrate lengthening principles

```
1 billboard
```

```
# A tibble: 317 × 79
```

	artist	track	date.entered	wk1	wk2	wk3	wk4	wk5	wk6	wk7	wk8
	<chr>	<chr>	<date>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1 2	Pac	Baby...	2000-02-26	87	82	72	77	87	94	99	NA
2 2	Ge+her	The ...	2000-09-02	91	87	92	NA	NA	NA	NA	NA
3 3	Doors D...	Kryp...	2000-04-08	81	70	68	67	66	57	54	53
4 3	Doors D...	Loser	2000-10-21	76	76	72	69	67	65	55	59
5 5	04 Boyz	Wobb...	2000-04-15	57	34	25	17	17	31	36	49
6 98	^0	Give...	2000-08-19	51	39	34	26	26	19	2	2
7 A*	Teens	Danc...	2000-07-08	97	97	96	95	100	NA	NA	NA
8 A	aliyah	I Do...	2000-01-29	84	62	51	41	38	35	35	38
9 A	aliyah	Try ...	2000-03-18	59	53	38	28	21	18	16	14
10 A	Adams, Yo...	Open...	2000-08-26	76	76	74	69	68	67	61	58

```
# i 307 more rows
```

Data lengthening

The `pivot_longer()` function requires minimum 3 arguments:

- `cols`: a vector of columns to be pivoted
- `names_to`: a character vector which specifies the name of the new column containing column NAMES from `cols` argument
- `values_to`: a character vector which specifies the name of the new column containing column VALUES from `cols` argument

```
1 billboard %>%  
2   pivot_longer(  
3     cols = starts_with("wk"),  
4     names_to = "week",  
5     values_to = "rank",  
6   )
```

Data lengthening

Result, long data:

```
# A tibble: 24,092 × 5
```

	artist	track		date.entered	week	rank
	<chr>	<chr>		<date>	<chr>	<dbl>
1	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk1	87
2	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk2	82
3	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk3	72
4	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk4	77
5	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk5	87
6	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk6	94
7	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk7	99
8	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk8	NA
9	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk9	NA
10	2 Pac	Baby Don't Cry (Keep...		2000-02-26	wk10	NA

```
# i 24,082 more rows
```

A lot of **NA** values in the resulting data; weeks when songs were not on billboard anymore - not really useful info - let's remove them.

Data lengthening

Removing `NA` values can be done using a handy optional argument `values_drop_na`:

```
1 billboard %>%
2   pivot_longer(
3     cols = starts_with("wk"),
4     names_to = "week",
5     values_to = "rank",
6     values_drop_na = TRUE
7   )
```

A tibble: 5,307 × 5

	artist	track	date.entered	week	rank
	<chr>	<chr>	<date>	<chr>	<dbl>
1	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk1	87
2	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk2	82
3	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk3	72
4	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk4	77
5	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk5	87
6	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk6	94
7	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk7	99
8	2Ge+her	The Hardest Part Of ...	2000-09-02	wk1	91
9	2Ge+her	The Hardest Part Of ...	2000-09-02	wk2	87
10	2Ge+her	The Hardest Part Of ...	2000-09-02	wk3	92

Data lengthening

Of course, `NA`s can also be removed using the already known `filter()` function:

```
1 billboard %>%
2   pivot_longer(
3     cols = starts_with("wk"),
4     names_to = "week",
5     values_to = "rank"
6   ) %>%
7   filter(!is.na(rank))
```

A tibble: 5,307 × 5

	artist	track	date.entered	week	rank
	<chr>	<chr>	<date>	<chr>	<dbl>
1	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk1	87
2	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk2	82
3	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk3	72
4	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk4	77
5	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk5	87
6	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk6	94
7	2 Pac	Baby Don't Cry (Keep...	2000-02-26	wk7	99
8	2Ge+her	The Hardest Part Of ...	2000-09-02	wk1	91
9	2Ge+her	The Hardest Part Of ...	2000-09-02	wk2	87
10	2Ge+her	The Hardest Part Of ...	2000-09-02	wk3	92

i 5,297 more rows

Data lengthening

We can further transform the obtained long data.

- For example, it would be useful if the `week` column would contain `1, 2, 3, ...` instead of `wk1, wk2, wk3, ...`
- Function `parse_number()`: finds the first number and keeps it, e.g. `wk12` transforms to `12`

```
1 billboard %>%
2   pivot_longer(
3     cols = starts_with("wk"),
4     names_to = "week",
5     values_to = "rank",
6     values_drop_na = TRUE
7   ) %>%
8   mutate(week = parse_number(week))
```

```
# A tibble: 5,307 × 5
  artist track date.entered week rank
  <chr> <chr> <date> <dbl> <dbl>
1 2 Pac Baby Don't Cry (Keep... 2000-02-26 1 87
2 2 Pac Baby Don't Cry (Keep... 2000-02-26 2 82
3 2 Pac Baby Don't Cry (Keep... 2000-02-26 3 72
4 2 Pac Baby Don't Cry (Keep... 2000-02-26 4 77
5 2 Pac Baby Don't Cry (Keep... 2000-02-26 5 87
6 2 Pac Baby Don't Cry (Keep... 2000-02-26 6 94
7 2 Pac Baby Don't Cry (Keep... 2000-02-26 7 99
8 2Ge+her The Hardest Part Of ... 2000-09-02 1 91
9 2Ge+her The Hardest Part Of ... 2000-09-02 2 87
10 2Ge+her The Hardest Part Of ... 2000-09-02 3 92
# i 5,297 more rows
```

Data lengthening

We can further transform the obtained long data.

- Finally, we could check how many songs stayed on top 100 billboard for more than a year (52 weeks)

```

1 billboard %>%
2   pivot_longer(
3     cols = starts_with("wk"),
4     names_to = "week",
5     values_to = "rank",
6     values_drop_na = TRUE
7   ) %>%
8   mutate(week = parse_number(week)) %>%
9   filter(week > 52) %>%
10  distinct(artist, track)

```

```

# A tibble: 4 × 2
  artist      track
  <chr>      <chr>
1 3 Doors Down Kryptonite
2 Creed       Higher
3 Hill, Faith Breathe
4 Lonestar    Amazed

```

Data widening

Data widening is less common than lengthening, but is sometimes used for presentation purposes and other special cases.

Data widening is done using the `pivot_wider()` function

country	year	cases
Angola	1999	800
Angola	2000	750
Angola	2001	925
Angola	2002	1020
India	1999	20100
India	2000	25650
India	2001	26800
India	2002	27255
Mongolia	1999	450
Mongolia	2000	512
Mongolia	2001	510
Mongolia	2002	586

country	1999	2000	2001	2002
Angola	800	750	925	1020
India	20100	25650	26800	27255
Mongolia	450	512	510	586

Pivot data wider

```
data %>%
  pivot_wider(
    names_from = "year",
    values_from = "cases"
  )
```

Source: Batra, Neale, et al. *The Epidemiologist R Handbook*. 2021

Data widening

We will use the `Orange` dataset to demonstrate widening principles

```
1 Orange
```

	Tree	age	circumference
1	1	118	30
2	1	484	58
3	1	664	87
4	1	1004	115
5	1	1231	120
6	1	1372	142
7	1	1582	145
8	2	118	33
9	2	484	69
10	2	664	111
11	2	1004	156
12	2	1231	172
13	2	1372	203

Data widening

The `pivot_wider()` function requires at least two arguments:

- `names_from`: specifies the column whose values will become the new column names
- `values_from`: specifies the column whose values will fill the new columns

```
1 Orange %>%  
2   pivot_wider(  
3     names_from = age,  
4     values_from = circumference  
5   )
```

Data widening

Result, wide data:

```
# A tibble: 5 × 8
  Tree `118` `484` `664` `1004` `1231` `1372` `1582`
  <ord> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 1      30     58     87     115     120     142     145
2 2      33     69    111     156     172     203     203
3 3      30     51     75     108     115     139     140
4 4      32     62    112     167     179     209     214
5 5      30     49     81     125     142     174     177
```

The column names now represent the `age` of the tree, but they are not descriptive enough for somebody else to understand what is going on.

Data widening

We can add a prefix for each new column name using the optional `names_prefix` argument:

```
1 Orange %>%
2   pivot_wider(
3     names_from = age,
4     values_from = circumference,
5     names_prefix = "age_days_"
6   )
```

```
# A tibble: 5 × 8
  Tree age_days_118 age_days_484 age_days_664 age_days_1004 age_days_1231
  <ord>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 1          30         58         87         115         120
2 2          33         69        111         156         172
3 3          30         51         75         108         115
4 4          32         62        112         167         179
5 5          30         49         81         125         142
# i 2 more variables: age_days_1372 <dbl>, age_days_1582 <dbl>
```

Data widening

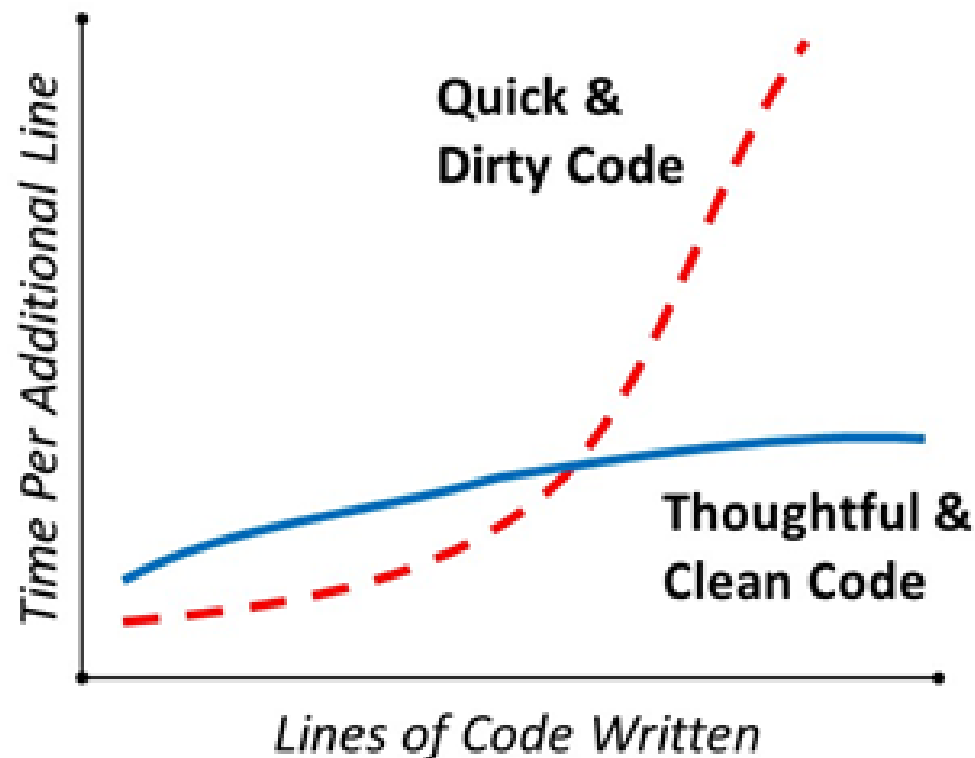
For complex names, we can use the optional `names_glue` argument (advanced), which uses a placeholder inside `{ }`:

```
1 Orange %>%
2   pivot_wider(
3     names_from = age,
4     values_from = circumference,
5     names_glue = "{age} [days]"
6   )
```

```
# A tibble: 5 × 8
  Tree `118 [days]` `484 [days]` `664 [days]` `1004 [days]` `1231 [days]`
  <ord>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>
1 1          30         58         87         115         120
2 2          33         69        111         156         172
3 3          30         51         75         108         115
4 4          32         62        112         167         179
5 5          30         49         81         125         142
# i 2 more variables: `1372 [days]` <dbl>, `1582 [days]` <dbl>
```

Tidy code

Messy code can run just as well as clean code. However, clean code makes life easier for you and others when it is revisited in the future.



Tidy code: spacing

Use spaces on both sides of mathematical operators (+, −, etc.), assignment operators (<-), and pipes (%>%).

```
1 # Strive for
2 z <- (a + b)^2 / d
3
4 # Avoid
5 z<-( a + b ) ^2/d
```

Do not put a space between a function name and its opening parenthesis. Put a space after commas, as in standard English.

```
1 # Strive for
2 median(x, na.rm = TRUE)
3
4 # Avoid
5 median (x ,na.rm=TRUE)
```

Tidy code: pipes

Pipes (`%>%`) should be surrounded by spaces and typically placed at the end of a line.

```
1 # Strive for
2 penguins %>%
3   filter(!is.na(sex), !is.na(body_mass)) %>%
4   count(sex)
5
6 # Avoid
7 penguins%>%filter(!is.na(sex), !is.na(body_mass))%>%count(sex)
```


Tidy code: pipes

If the function you're piping into has named arguments (like `mutate()` or `summarize()`), put each argument on a new line. If the function doesn't have named arguments (like `select()` or `filter()`), keep everything on one line unless it doesn't fit, in which case you should put each argument on its own line.

```
1 # Strive for
2 penguins %>%
3   group_by(sex) %>%
4   summarize(
5     avg_mass = mean(body_mass, na.rm = TRUE),
6     n = n()
7   )
8
9 # Avoid
10 penguins %>%
11   group_by(
12     sex
13   ) %>%
14   summarize(avg_mass = mean(body_mass, na.rm = TRUE), n = n())
```

Tidy code: pipes

Some of these rules can be relaxed if your pipeline fits easily on one line.

```
1 # This fits compactly on one line
2 df %>% mutate(y = x + 1)
3
4 # While this takes more lines, it is easily extended to
5 # more variables and more steps in the future
6 df %>%
7   mutate(
8     y = x + 1
9   )
```

Tidy code: ggplot2

The same rules apply as for pipes: treat `+` the same way as `%>%`.

```
1 penguins %>%
2   filter(!is.na(body_mass)) %>%
3   ggplot(aes(x = body_mass, y = flipper_len)) +
4   geom_point() +
5   geom_smooth(
6     method = "lm",
7     linewidth = 0.8,
8     color = "pink"
9   ) +
10  theme_minimal()
```

Tidy code: sectioning

As scripts get longer, *sectioning* makes them easier to navigate.

Create a section using `Ctrl` + `Shift` + `R` on Windows or `⌘` + `Shift` + `R` on macOS.

```
1 # Load data -----  
2  
3 # Plot data -----
```